

RESTAURANT ORDER MANAGEMENT

ID: FT0001

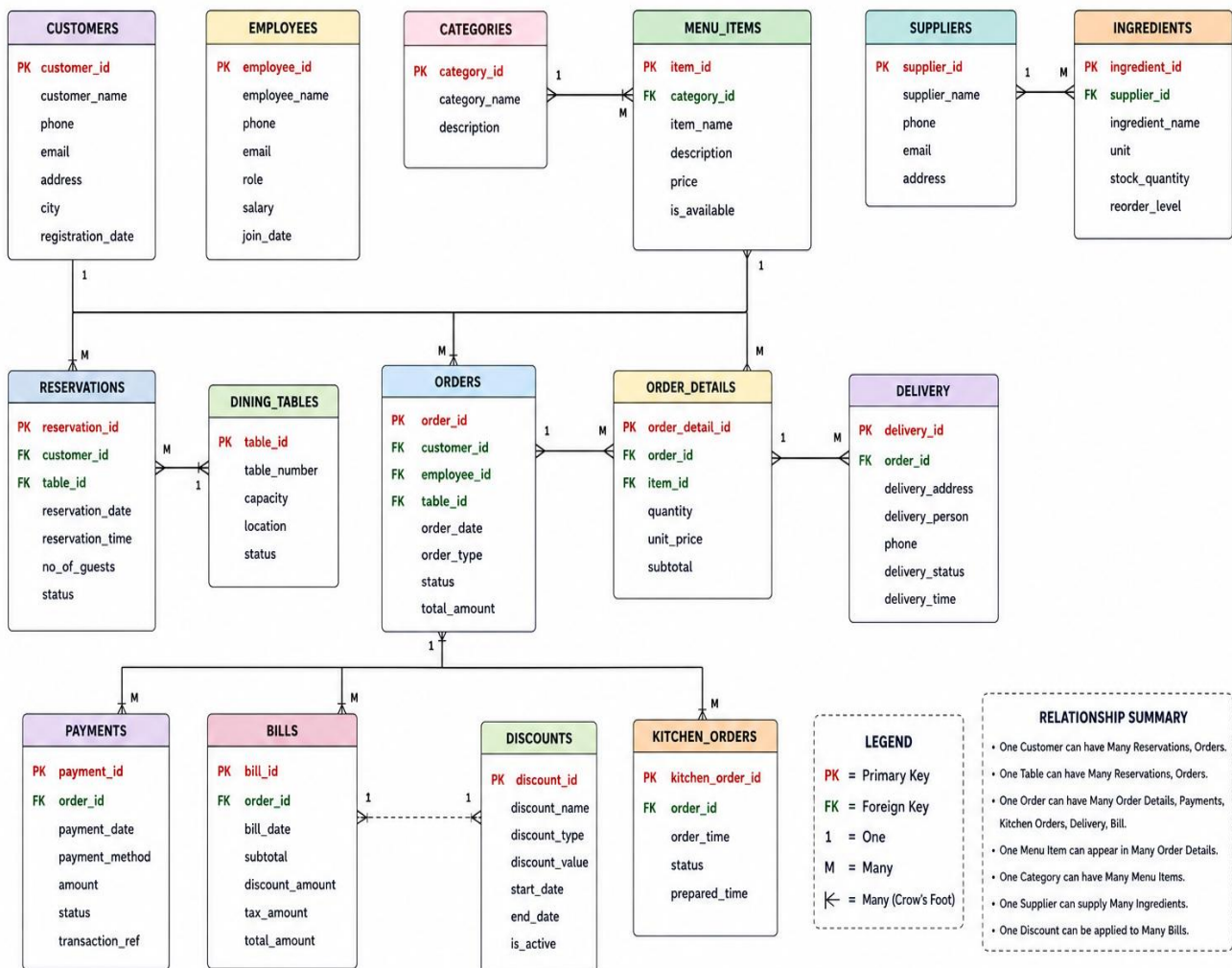
Name: Sowmiya B

College: Paavai Engineering College

GitHub URL: <https://github.com/sowmiya3904/Restaurant-Order-Management-SQL-Project.git>

Table Schema:

RESTAURANT ORDER MANAGEMENT SYSTEM – ER DIAGRAM



Basic Level

- Display the names and phone numbers of all customers.
- Display all menu items with a price greater than 300.
- Display all customers who belong to Chennai.
- Display all orders placed on 2025-06-10.
- Display all payment records with the status Paid.
- Display all payment records with the status Pending.
- Display all delivery records with the status Delivered.
- Display all kitchen orders whose preparation status is Pending.
- Display all suppliers located in Coimbatore.
- Display all ingredients whose stock quantity is greater than the reorder level.
- Display only the names of all suppliers.
- Display only the names of all ingredients.
- Display all menu items in ascending order of price.
- Display all menu items in descending order of price.
- Count the total number of customers.
- Count the total number of orders.
- Count the total number of menu items.
- Find the maximum price of a menu item.
- Find the minimum price of a menu item.
- Find the average price of all menu items.
- Find the total amount received through payments.
- Display the distinct cities from the Customers table.

- Display all menu items whose price is between 200 and 500.
- Display the top 5 most expensive menu items.
- Display all bills sorted by the final amount in descending order.
- Display all suppliers sorted alphabetically by supplier name.
- Display all customers whose names start with the letter A.
- Display all customers whose city ends with the letter i.
- Display all menu items whose name contains the word Chicken.
- Display all employees whose salary is greater than 40,000.
- Display all employees working as Manager.
- Display all employees working as Chef.
- Display all orders whose total amount is less than 500.
- Display all employees whose salary is between 25,000 and 40,000.
- Display all suppliers from Chennai and Madurai.
- Display all ingredients measured in kilograms (Kg).
- Display all ingredients whose reorder level is less than 10.

INTERMEDIATE LEVEL

- Display the customer's name along with their order ID and order date.
- Display the customer's name and payment amount.
- Display the customer's name, final bill amount, and tax.
- Display the customer's name and delivery person.
- Display the menu item name and quantity ordered.
- Display the category name along with menu items.
- Display the chef's name along with the order ID.
- Display the total payment received by each payment method.
- Count the number of payments made by each payment method.

- Display the total number of customers in each city.
- Display the average salary (rounded) for each designation.
- Display the total stock quantity of ingredients grouped by unit. Display payment methods having more than 10 transactions.
- Display cities having more than one supplier.
- Find the highest bill amount.
- Find the lowest bill amount.
- Display customers whose bill amount is greater than 500.
- Display the total amount spent by each customer.
- Display the number of orders placed by each customer.
- Display all customers who have made payments using UPI.
- Display the customer's name, payment method, and payment status.
- Display the customer's name, delivery status, and delivery person.
- Display menu item names with their category names.
- Display the total number of menu items in each category.
- Display the average menu price in each category.
- Display the highest-priced menu item in each category.
- Display all orders along with the chef handling them.
- Display the chef's name and preparation status.
- Display all orders with their payment status.
- Display customers who paid using Cash.
- Display customers who paid using Card.
- Display customers whose orders have been delivered.
- Display customers whose kitchen orders are completed.
- Display the total number of orders handled by each chef.
- Display the number of ingredients available for each unit of measurement.
- Display each payment method along with its maximum and minimum payment amount.
- Display customers in alphabetical order along with their bill amount.
- Display customers whose bill amount is between 500 and 800.
- Display the average bill amount rounded to two decimal places.
- Display the average payment amount rounded to the nearest integer.

- Display the average menu item price rounded to one decimal place.
- Display the floor value of the average bill amount.
- Display the floor value of the average payment amount.
- Display the ceiling value of the average menu item price.
- Display each bill amount truncated to one decimal place.
- Display each payment amount truncated to zero decimal places.
- Display each menu item's original price along with its rounded, floor, and ceiling values.
- Display the average bill amount using ROUND (), FLOOR (), and CEILING () functions.
- Increase the price of all menu items by 10%.

ADVANCED LEVEL

Display the customer(s) who paid the highest bill.

```
SELECT c.customer_name, b.final_amount
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id
JOIN Bills b
ON o.order_id = b.order_id
WHERE b.final_amount =(
SELECT MAX(final_amount)
FROM Bills
);
```

#Display customers whose bill amount is greater than the average bill.

```
SELECT c.customer_name, b.final_amount
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id
JOIN Bills b
```

```
ON o.order_id = b.order_id
WHERE b.final_amount >
(
SELECT AVG(final_amount)
FROM Bills
);
```

```
#Display the second highest bill amount
SELECT MAX(final_amount) AS Second_Highest
FROM Bills
WHERE final_amount <
(
SELECT MAX(final_amount)
FROM Bills
);
```

```
#Display menu items whose price is greater than the average menu price.
SELECT item_name, price
FROM Menu_Items
WHERE price >
(
SELECT AVG(price)
FROM Menu_Items
);
```

```
#Display customers who have placed more than one order.
SELECT customer_id,
COUNT(order_id) AS Total_Orders
FROM Orders
GROUP BY customer_id
HAVING COUNT(order_id) > 1;
```

#Rank customers based on their bill amount.

```
SELECT bill_id,  
final_amount,  
RANK() OVER(ORDER BY final_amount DESC) AS Bill_Rank  
FROM Bills;
```

#Assign row numbers to all menu items based on price.

```
SELECT item_name,  
price,  
ROW_NUMBER() OVER(ORDER BY price DESC) AS Row_Num  
FROM Menu_Items;
```

#Display the dense rank of payments based on amount.

```
SELECT payment_id,amount,  
DENSE_RANK() OVER(ORDER BY amount DESC) AS Payment_Rank  
FROM Payments;
```

#Create a View to display customer bills.

```
CREATE VIEW Customer_Bill_View AS  
SELECT c.customer_name,  
o.order_id,  
b.final_amount  
FROM Customers c  
JOIN Orders o  
ON c.customer_id=o.customer_id  
JOIN Bills b  
ON o.order_id=b.order_id;
```

```
SELECT * FROM Customer_Bill_View;
```

#Create a Stored Procedure to display all paid payments.

```
DELIMITER $$
```

```
CREATE PROCEDURE PaidPayments()
BEGIN
SELECT *
FROM Payments
WHERE payment_status='Paid';
END $$
```

```
DELIMITER ;
CALL PaidPayments();
```

#Create a Stored Procedure to display all customers from a given city.

```
DELIMITER $$
```

```
CREATE PROCEDURE CustomerCity(IN city_name VARCHAR(30))
BEGIN
SELECT *
FROM Customers
WHERE city = city_name;
END $$
```

```
DELIMITER ;
CALL CustomerCity('Chennai');
```

#Create a Trigger to prevent negative bill amounts.

```
DELIMITER $$
```

```
CREATE TRIGGER CheckBill
BEFORE INSERT
ON Bills
FOR EACH ROW
BEGIN
```

```
IF NEW.final_amount < 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT='Invalid Bill Amount';
END IF;
END $$
```

```
DELIMITER ;
```

```
#Use a Common Table Expression (CTE) to display bills greater than ₹500.
WITH BillData AS
(
SELECT bill_id,
final_amount
FROM Bills
)
SELECT *
FROM BillData
WHERE final_amount > 500;
```

```
#Classify payments based on payment method using CASE.
SELECT payment_id,
payment_method,
CASE
WHEN payment_method='Cash' THEN 'Offline'
WHEN payment_method='Card' THEN 'Online'
WHEN payment_method='UPI' THEN 'Online'
END AS Payment_Type
FROM Payments;
```

```
#Save a transaction using COMMIT.
START TRANSACTION;
```

```
UPDATE Bills
SET discount = discount + 20
WHERE bill_id = 801;
```

```
COMMIT;
```

```
#Display customers who have placed orders.
```

```
SELECT customer_name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
);
```

```
#Display menu items costing more than the average price.
```

```
SELECT item_name, price
FROM Menu_Items
WHERE price > (
    SELECT AVG(price)
    FROM Menu_Items
);
```

```
#Find the highest payment amount.
```

```
SELECT *
FROM Payments
WHERE amount = (
    SELECT MAX(amount)
    FROM Payments
);
```

```
#Find employees earning more than the average salary.
```

```
SELECT employee_name, salary
```

```
FROM Employees
WHERE salary > (
    SELECT AVG(salary)
    FROM Employees
);
```

```
#Find suppliers supplying ingredients costing more than ₹100.
SELECT supplier_name
FROM Suppliers
WHERE supplier_id IN (
    SELECT supplier_id
    FROM Ingredients
    WHERE stock_quantity > 100
);
```

```
#Find customers who made payments greater than ₹500.
SELECT customer_name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
    WHERE order_id IN (
        SELECT order_id
        FROM Payments
        WHERE amount > 500
    )
);
```